

Computational Lab: Optimization Methods in Python

ISE 5406 — Virginia Tech, Spring 2026

Dr. Robert Hildebrand

Lab Overview

In this lab you will:

1. Implement **golden section search** (derivative-free, 1D) from scratch
2. Implement **bisection search** (first-derivative, 1D) from scratch
3. Compare their convergence rates experimentally
4. Use `scipy.optimize` for multivariate optimization: derivative-free, first-order, and second-order methods
5. Visualize optimization trajectories on contour plots

1 Background: Three Classes of Optimization Methods

Optimization methods are classified by the **derivative information** they use:

Derivative-Free (Zero-Order) Methods

Use **only function values** $f(\mathbf{x})$. No gradients or Hessians required.

When to use: Black-box functions, simulation-based optimization, noisy objectives, non-differentiable functions.

Examples: Golden section search (1D), Nelder–Mead simplex, Powell’s method, genetic algorithms.

Convergence: Typically *sublinear* or *linear*. Slow but robust.

Scipy: `minimize(f, x0, method='Nelder-Mead')` or `method='Powell'`

First-Order (Gradient) Methods

Use **function values and gradients** $f(\mathbf{x}), \nabla f(\mathbf{x})$.

When to use: Smooth objectives where gradients are available or cheap to compute.

Examples: Steepest descent, conjugate gradient, BFGS (quasi-Newton), L-BFGS.

Convergence: *Linear* (steepest descent) to *superlinear* (BFGS). Much faster than derivative-free.

Scipy: `minimize(f, x0, jac=grad_f, method='CG')` or `method='BFGS'`

Second-Order (Newton) Methods

Use **function values, gradients, and Hessians** $f(\mathbf{x}), \nabla f(\mathbf{x}), \nabla^2 f(\mathbf{x})$.

When to use: Medium-scale problems where Hessian is available or approximated well.

Examples: Newton’s method, Newton–CG, trust-region Newton.

Convergence: *Quadratic* (near solution). Fastest, but each iteration is expensive.

Scipy: `minimize(f, x0, jac=grad_f, hess=hess_f, method='Newton-CG')` or `method='trust-ncg'`

Property	Derivative-Free	First-Order	Second-Order
Info used	$f(\mathbf{x})$	$f(\mathbf{x}), \nabla f(\mathbf{x})$	$f(\mathbf{x}), \nabla f(\mathbf{x}), \nabla^2 f(\mathbf{x})$
Cost per iteration	Low	Medium	High ($O(n^3)$)
Convergence rate	Sublinear/Linear	Linear/Superlinear	Quadratic
Robustness	High	Medium	Needs PD Hessian
Scalability	Low ($n \lesssim 10^2$)	High ($n \lesssim 10^6$)	Medium ($n \lesssim 10^4$)

2 Part 1: Golden Section Search (Derivative-Free, 1D)

Golden section search finds the minimum of a **unimodal** function on $[a, b]$ using only function evaluations.

Algorithm

- Let $\varphi = \frac{1+\sqrt{5}}{2} \approx 1.618$ (golden ratio), and $\rho = 2 - \varphi \approx 0.382$.
1. Set $\lambda = a + \rho(b - a)$ and $\mu = a + (1 - \rho)(b - a)$.
 2. Evaluate $f(\lambda)$ and $f(\mu)$.
 3. If $f(\lambda) > f(\mu)$: set $a \leftarrow \lambda$ (eliminate left portion).
If $f(\lambda) \leq f(\mu)$: set $b \leftarrow \mu$ (eliminate right portion).
 4. Recompute only *one* new interior point (the other is reused).
 5. Repeat until $|b - a| < \epsilon$.

Key properties:

- Each iteration reduces the interval by factor $1 - \rho = \varphi - 1 \approx 0.618$.
- Only **1 new function evaluation** per iteration (after the first).
- **Convergence rate:** Linear with ratio $r = 0.618$. After k iterations: $|b_k - a_k| = 0.618^k |b_0 - a_0|$.

```
def golden_section(f, a, b, tol=1e-8, max_iter=100):
    phi = (1 + 5**0.5) / 2
    rho = 2 - phi # ~0.382
    lam = a + rho * (b - a)
    mu = a + (1 - rho) * (b - a)
    f_lam, f_mu = f(lam), f(mu)
    history = [(a, b)]
    for _ in range(max_iter):
        if abs(b - a) < tol:
            break
        if f_lam > f_mu:
            a = lam
            lam, f_lam = mu, f_mu
            mu = a + (1 - rho) * (b - a)
            f_mu = f(mu)
        else:
            b = mu
            mu, f_mu = lam, f_lam
            lam = a + rho * (b - a)
            f_lam = f(lam)
        history.append((a, b))
    return (a + b) / 2, history
```

3 Part 2: Bisection Search (First-Derivative, 1D)

Bisection search finds the minimum of a differentiable **unimodal** function by finding where $f'(x) = 0$, using the sign of the derivative.

Algorithm

Require: $f'(a) < 0$ and $f'(b) > 0$ (function decreasing at a , increasing at b).

1. Set $m = \frac{a+b}{2}$.
2. Evaluate $f'(m)$.
3. If $f'(m) < 0$: set $a \leftarrow m$ (minimum is to the right).
If $f'(m) > 0$: set $b \leftarrow m$ (minimum is to the left).
If $f'(m) = 0$: done, m is the minimizer.
4. Repeat until $|b - a| < \epsilon$.

Key properties:

- Each iteration **halves** the interval exactly: factor 0.5.
- Requires **1 derivative evaluation** per iteration.
- **Convergence rate:** Linear with ratio $r = 0.5$. After k iterations: $|b_k - a_k| = 0.5^k |b_0 - a_0|$.
- **Faster than golden section** ($0.5 < 0.618$), but requires derivative access.

```
def bisection_search(df, a, b, tol=1e-8, max_iter=100):
    """Find x* where df(x*) = 0 using bisection on the derivative."""
    history = [(a, b)]
    for _ in range(max_iter):
        if abs(b - a) < tol:
            break
        m = (a + b) / 2
        if df(m) < 0:
            a = m          # minimum is to the right
        elif df(m) > 0:
            b = m          # minimum is to the left
        else:
            a, b = m, m    # found exact root
        history.append((a, b))
    return (a + b) / 2, history
```

Convergence Rate Comparison

Method	Info Used	Evals/Iter	Reduction Factor	Iters for 10^{-8}
Golden Section	$f(x)$	1 function	0.618	≈ 38
Bisection	$f'(x)$	1 derivative	0.500	≈ 27

Using derivative information gives $\sim 30\%$ fewer iterations. The trade-off: derivatives may be expensive or unavailable.

4 Part 3: Scipy Optimization Methods

The `scipy.optimize.minimize` function provides a unified interface to many optimization algorithms:

```
from scipy.optimize import minimize

result = minimize(fun, x0, method=..., jac=..., hess=..., options={...})
# result.x          -> optimal point
# result.fun        -> optimal value
# result.nit        -> number of iterations
# result.nfev       -> number of function evaluations
# result.success    -> True if converged
```

Derivative-Free Methods

```
# Nelder-Mead: simplex-based, no derivatives needed
result = minimize(f, x0, method='Nelder-Mead')

# Powell: direction set method, no derivatives needed
result = minimize(f, x0, method='Powell')
```

First-Order Methods

```
# Conjugate Gradient: first-order, good for large problems
result = minimize(f, x0, jac=grad_f, method='CG')

# BFGS: quasi-Newton, superlinear convergence
result = minimize(f, x0, jac=grad_f, method='BFGS')

# L-BFGS-B: limited-memory BFGS, for very large problems
result = minimize(f, x0, jac=grad_f, method='L-BFGS-B')
```

Second-Order Methods

```
# Newton-CG: Newton's method with CG inner solver
result = minimize(f, x0, jac=grad_f, hess=hess_f, method='Newton-CG')

# Trust-region Newton: globally convergent Newton
result = minimize(f, x0, jac=grad_f, hess=hess_f, method='trust-ncg')
```

Test Functions

Course function: $f(x_1, x_2) = x_1^2 + 4x_2^2$

Convex quadratic, condition number $\kappa = 4$, optimum at $(0, 0)$.

Rosenbrock: $f(x_1, x_2) = (1 - x_1)^2 + 100(x_2 - x_1^2)^2$

Classic benchmark, narrow curved valley, optimum at $(1, 1)$. Condition number ~ 2500 at solution. Tests how well methods handle ill-conditioning.

Beale: $f(x_1, x_2) = (1.5 - x_1 + x_1x_2)^2 + (2.25 - x_1 + x_1x_2^2)^2 + (2.625 - x_1 + x_1x_2^3)^2$

Non-quadratic, optimum at $(3, 0.5)$. Tests robustness to nonlinearity.

5 Part 4: Exercises

Complete the exercises in the accompanying Jupyter notebook `Lab-Optimization.ipynb`.

Exercise 1: Golden Section vs. Bisection. Run both methods on $f(x) = (x - 3)^2 + 1$ on $[0, 10]$. Plot the interval width $|b_k - a_k|$ vs. iteration number on a **log scale**. Verify that:

- Golden section: slope $\approx \log_{10}(0.618) \approx -0.209$
- Bisection: slope $\approx \log_{10}(0.5) \approx -0.301$

Exercise 2: Method Comparison on a Quadratic. Minimize $f(x_1, x_2) = x_1^2 + 4x_2^2$ from $x_0 = (4, 2)^T$ using Nelder–Mead, BFGS, and Newton–CG. Compare: number of iterations, function evaluations, and final accuracy.

Exercise 3: Rosenbrock Challenge. Minimize the Rosenbrock function from $x_0 = (-1, 1)^T$ using all three method classes. Plot the optimization trajectories on contour plots. Which method class handles the narrow valley best?

Exercise 4: Convergence Rate Analysis. For the quadratic $f(x_1, x_2) = x_1^2 + 4x_2^2$, record $\|x_k - x^*\|$ at each iteration for steepest descent (CG with 1 restart), BFGS, and Newton–CG. Plot on a log scale and estimate the convergence rates (linear, superlinear, quadratic).

Exercise 5: Effect of Condition Number. Minimize $f(\mathbf{x}) = x_1^2 + \kappa x_2^2$ for $\kappa \in \{1, 10, 100, 1000\}$ using BFGS. How does the number of iterations depend on κ ? Does this match the theory ($O(\kappa \log(1/\epsilon))$ for steepest descent)?

Takeaway

- **Derivative-free** methods are the most robust but slowest. Use when derivatives are unavailable.
- **First-order** methods (especially BFGS) offer the best balance of speed and robustness for most problems.
- **Second-order** methods converge fastest near the solution but require Hessian computation and may struggle far from the optimum.
- The **condition number** κ controls how hard a problem is for gradient methods. Newton’s method is largely immune to ill-conditioning.